



17. 캐릭터

게임에 생기를 불어넣는 기술

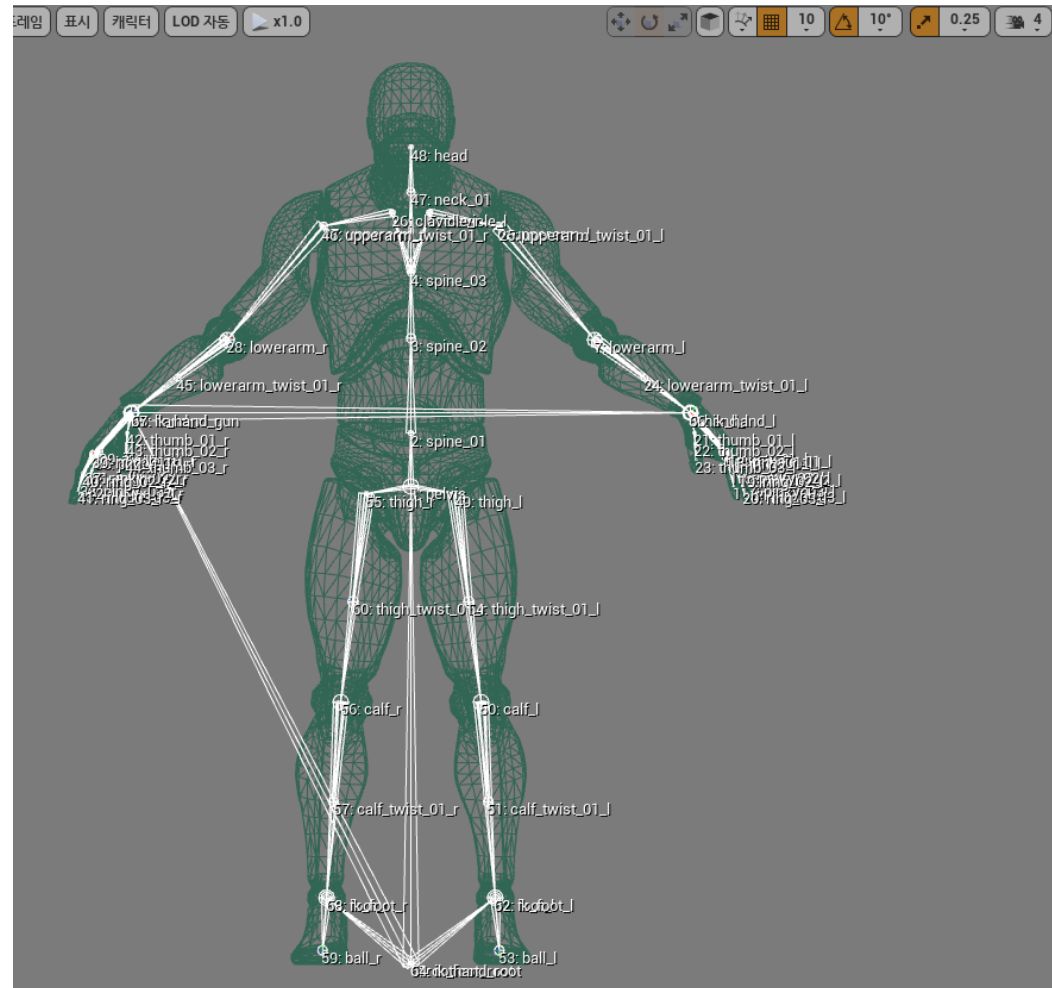




스켈레탈 애니메이션을 구성하는 본과 메시

본 구성 정보

- ◆ 본 이름
- ◆ 바인드포즈 (Bindpose)
 - 최초 배치 정보
 - 본 애니메이션 기준점
- ◆ 트랜스폼 정보
 - 시간 기반





```
class Bone
{
private:
    std::size_t _Hash = 0;

    // 본의 이름
    std::string _Name;

    // 본의 현재 트랜스폼 정보
    Transform _Transform;

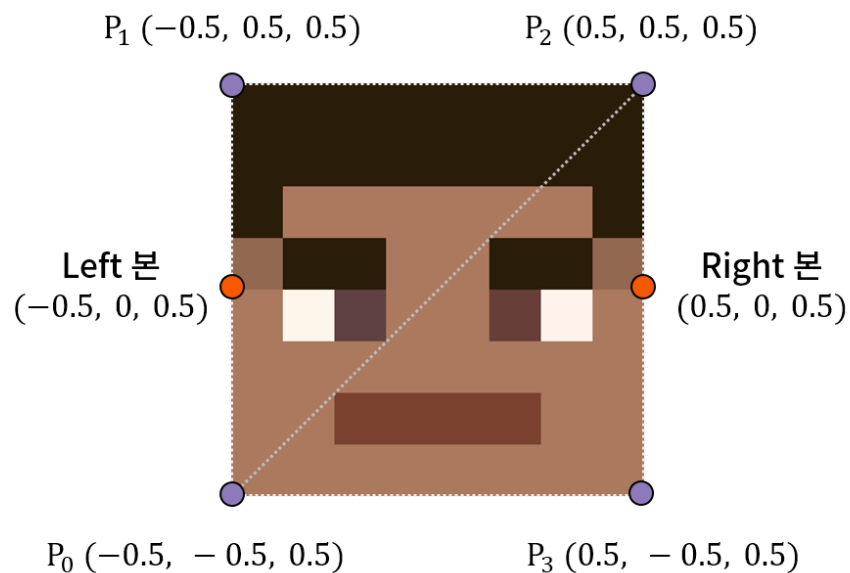
    // 본의 최초 트랜스폼 정보
    Transform _BindPose;
};
```





● 본을 추가한 메시 구조

```
enum class MeshType : UINT32
{
    Normal = 0,
    Skinned
};
```



정점 버퍼

인덱스	좌표	UV좌표
0	$(-0.5, -0.5, 0.5)$	$(0.125, 0.75)$
1	$(-0.5, 0.5, 0.5)$	$(0.125, 0.875)$
2	$(0.5, 0.5, 0.5)$	$(0.25, 0.75)$
3	$(0.5, -0.5, 0.5)$	$(0.25, 0.875)$

인덱스 버퍼

인덱스	삼각형	정점인덱스
0	0	0
1	0	1
2	0	2
3	1	0
4	1	2
5	1	3

본의 목록

이름	좌표
Left	$(-0.5, 0, 0.5)$
Right	$(0.5, 0, 0.5)$





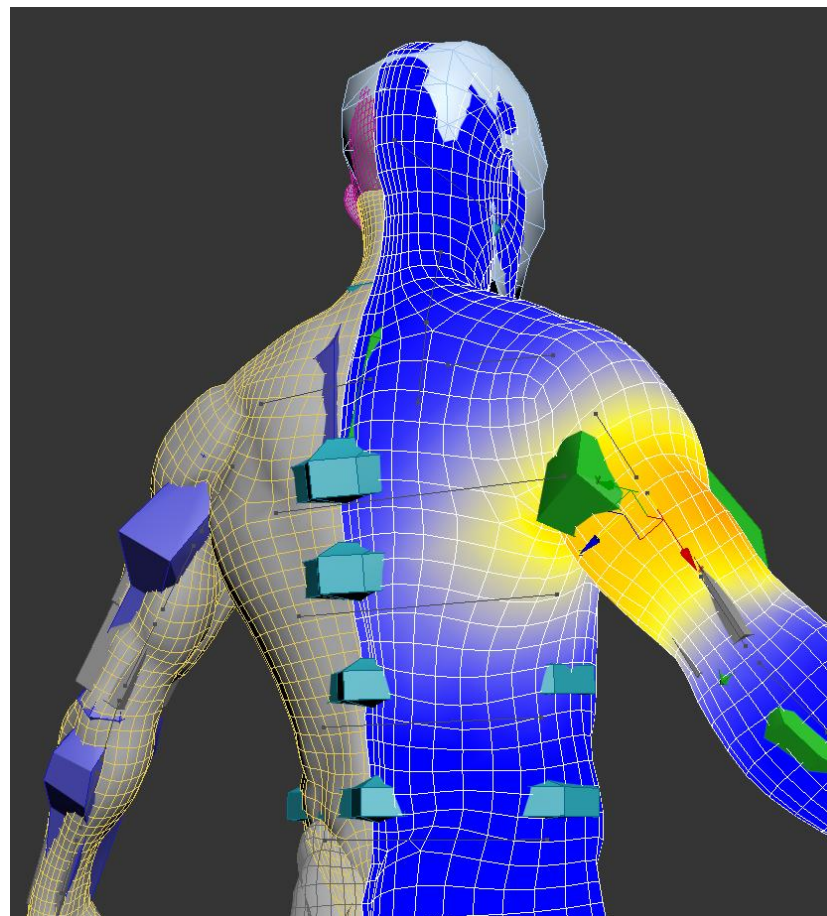
- 리깅

- ◆ Bone과 정점 연결
- ◆ Bone의 움직임에 따라 Mesh가 변형되도록 설정

- 수학적 관점

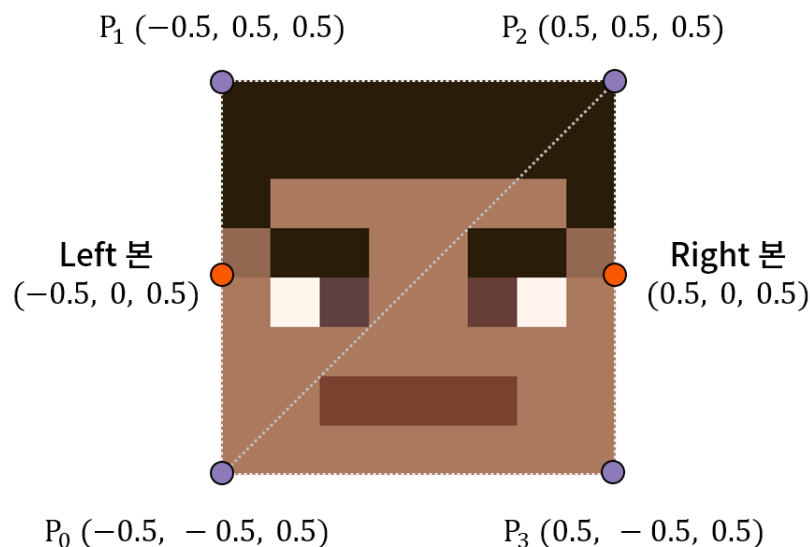
- ◆ 정점에 Bone Transform과 Weight 설정

```
struct Weight
{
    std::vector<std::string> Bones;
    std::vector<float> Values;
};
```





● 점과 본과의 연결 정보를 추가한 메시 구조의 예



정점 버퍼

인덱스	좌표	UV좌표	연결된 본의 수	연결 정보
0	(-0.5, -0.5, 0.5)	(0.125, 0.75)	1	Left, 1
1	(-0.5, 0.5, 0.5)	(0.125, 0.875)	1	Left, 1
2	(0.5, 0.5, 0.5)	(0.25, 0.75)	1	Right, 1
3	(0.5, -0.5, 0.5)	(0.25, 0.875)	1	Right, 1

인덱스 버퍼

인덱스	삼각형	정점인덱스
0	0	0
1	0	1
2	0	2
3	1	0
4	1	2
5	1	3

본의 목록

이름	좌표
Left	(-0.5, 0, 0.5)
Right	(0.5, 0, 0.5)

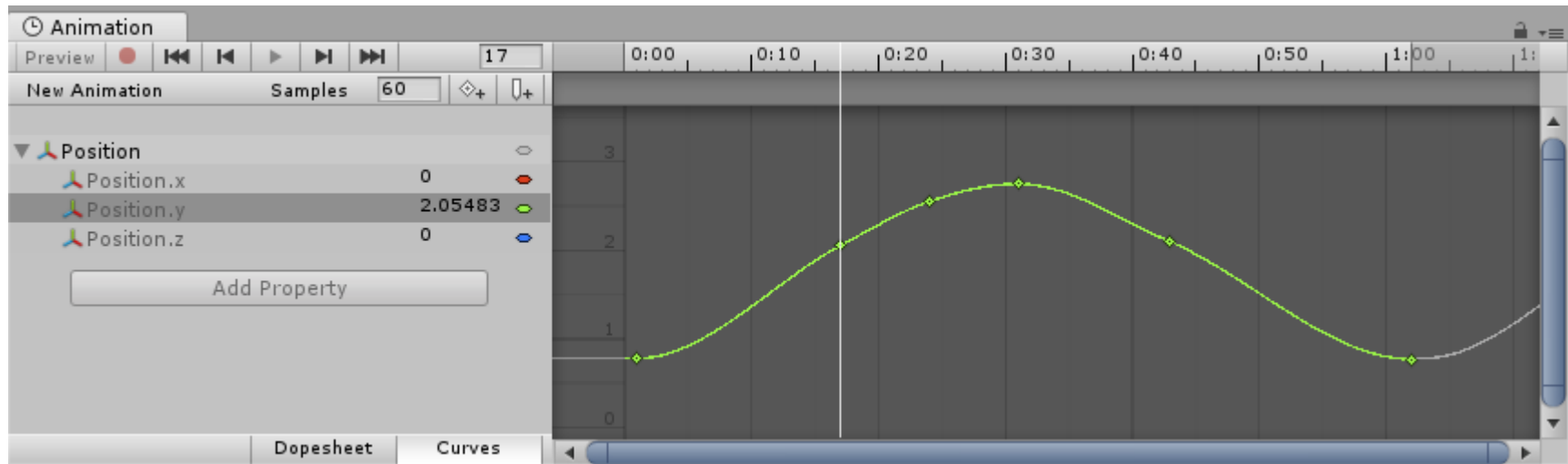
```
bones = {  
    {"left", Bone("left", Transform(Vector3(-1.f, 0.f, 0.f) * halfSize))},  
    {"right", Bone("right", Transform(Vector3(1.f, 0.f, 0.f) * halfSize))}  
};
```

```
weights = {  
    { {"left"}, {1.f} }, { {"right"}, {1.f} },  
    { {"left"}, {1.f} }, { {"right"}, {1.f} }  
};
```



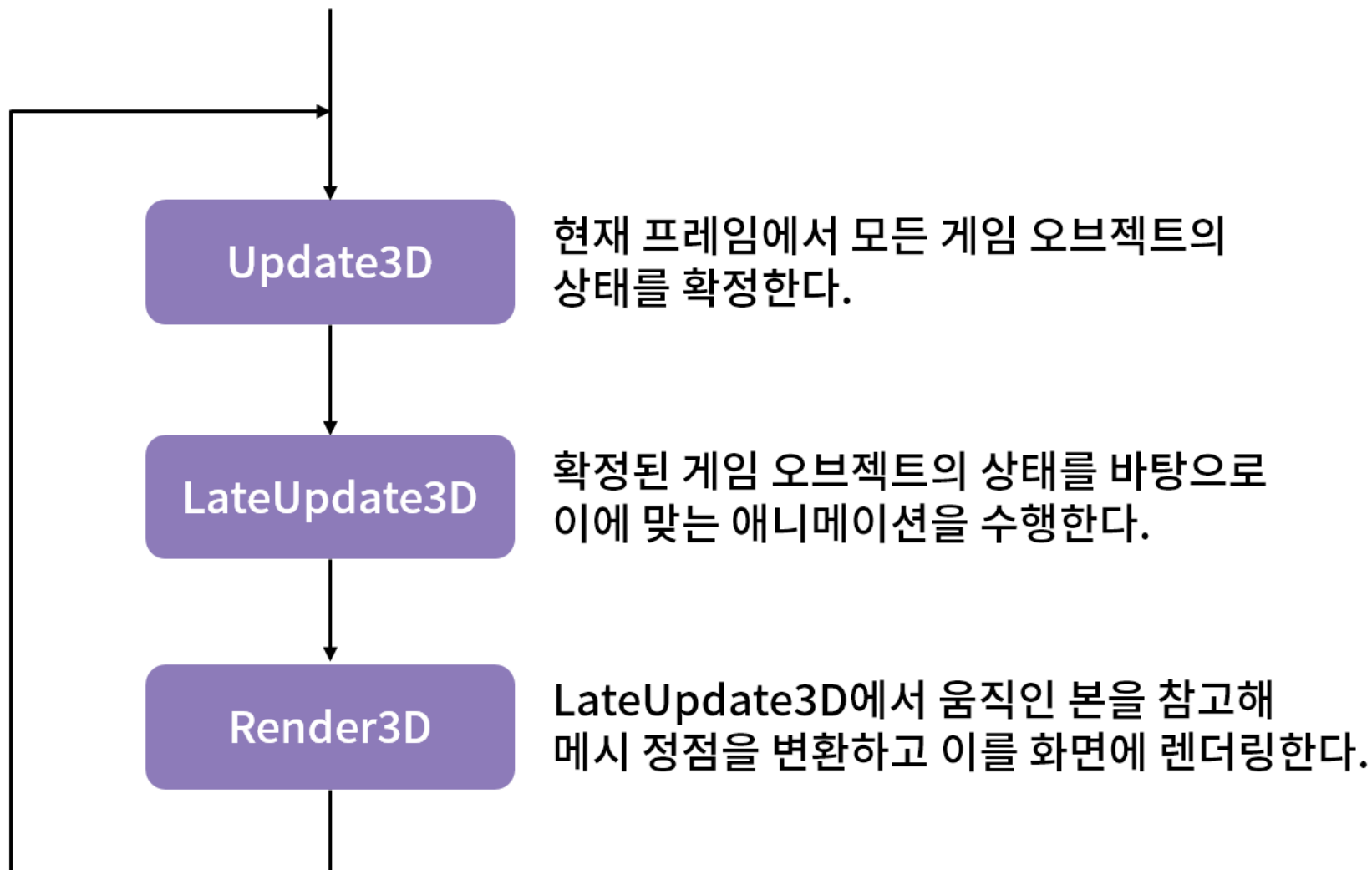


- 키프레임 Keyframe
 - ◆ 애니메이션에서
특정 시간의 Bone Transform 상태를 저장한 기준 프레임
- 키프레임 데이터 예시





- 애니메이션 워크플로우





```
for (size_t vi = 0; vi < vertexCount; ++vi)
{
    vertices[vi].Position = Vector4(InMesh.GetVertices()[vi]);

    if (InMesh.IsSkinnedMesh())
    {
        Vector3 totalDeltaPosition;
        Weight w = InMesh.GetWeights()[vi];
        for (size_t wi = 0; wi < InMesh.GetConnectedBones()[vi]; ++wi)
        {
            std::string boneName = w.Bones[wi];
            if (InMesh.HasBone(boneName))
            {
                const Transform& boneTransform
                    = InMesh.GetBone(boneName).GetTransform();

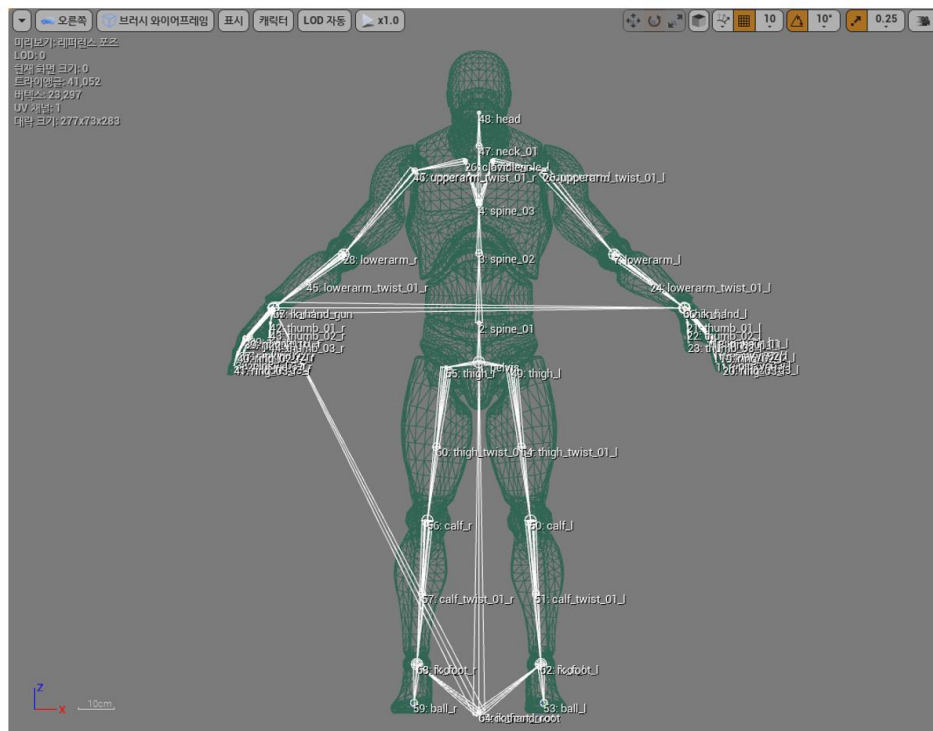
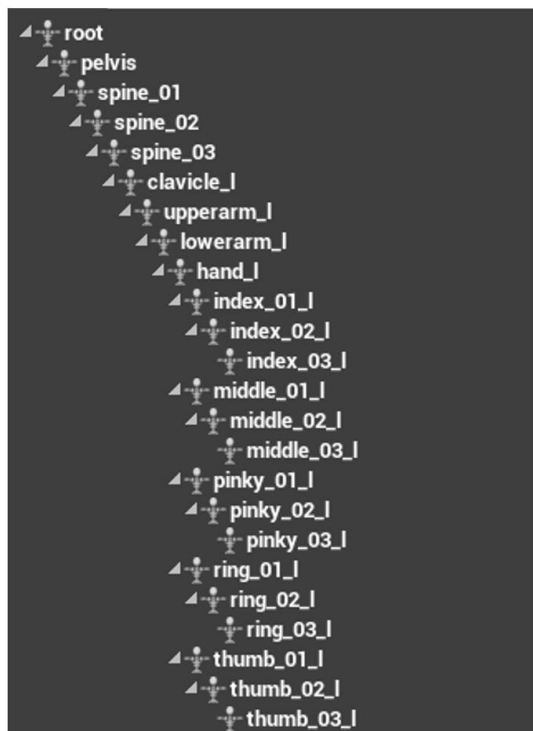
                Vector3 deltaPosition = boneTransform.GetPosition()
                    - InMesh.GetBindPose(boneName).GetPosition();
                totalDeltaPosition += deltaPosition * w.Values[wi];
            }
        }

        vertices[vi].Position += Vector4(totalDeltaPosition, false);
    }
}
```





- 본 계층 구조 Bone Hierarchy
 - ◆ Bone들이 부모-자식 관계로 연결된 트리 구조
 - ◆ 부모 Bone의 Transform은 자식 Bone에 전달
 - ◆ 자식 Bone의 Transform은 부모 Bone에 영향 없음
 - ◆ 부모 Bone은 여러 자식 Bone을 가질 수 있음
 - ◆ 자식 Bone은 하나의 부모 Bone만 가짐





```
class TransformComponent
{
private:
    // 계층 구조를 위한 변수
    Transform _LocalTransform;
    Transform _WorldTransform;

    // 부모, 자식 노드
    TransformComponent* _ParentPtr = nullptr;
    std::vector<TransformComponent*> _ChildrenPtr;
};
```



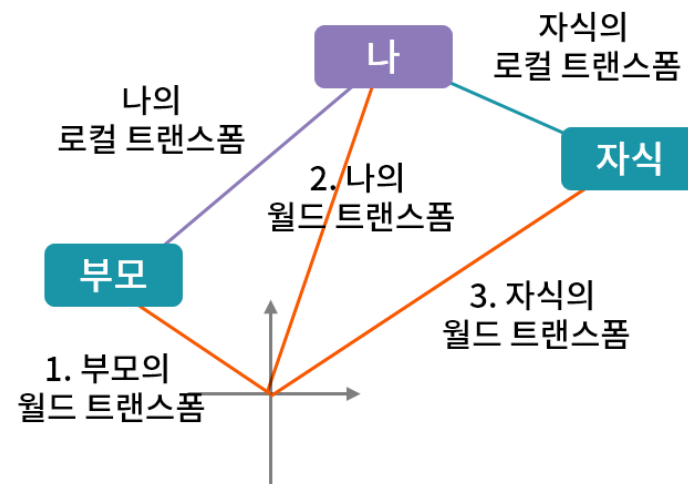
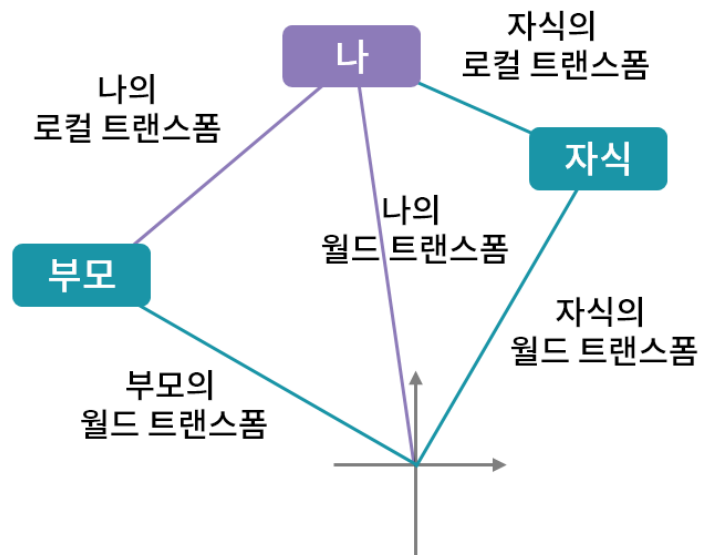


- 로컬 트랜스폼 / 월드 트랜스폼 구분 → 다양한 상황에 대처
- 로컬 트랜스폼
 - ◆ 부모로부터 상대적인 트랜스폼
 - ◆ 상대 트랜스폼 (Relative transform)
- 월드 트랜스폼
 - ◆ 계층 구조와 상관 없이 자신의 속한 공간에서의 트랜스폼
 - ◆ 절대 트랜스폼 (Absolute transform)



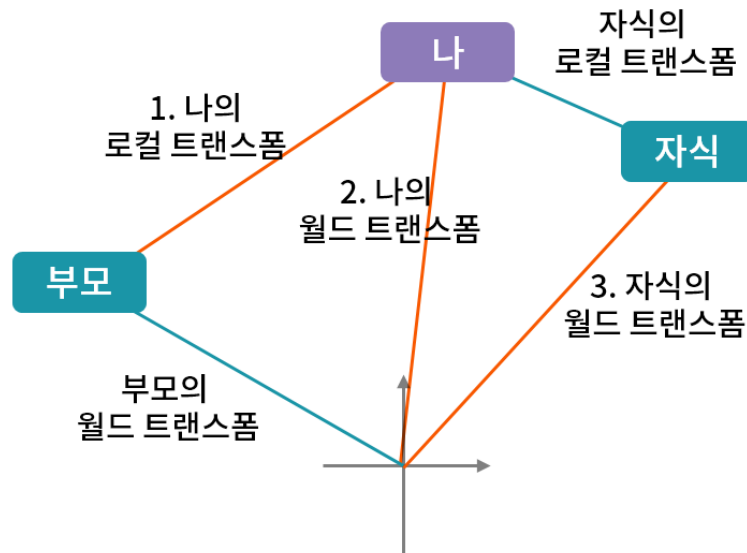
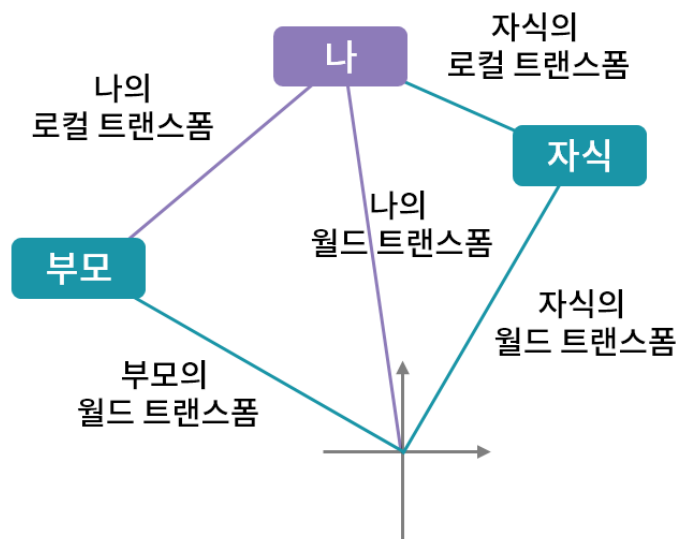


- 부모의 월드 트랜스폼이 변경



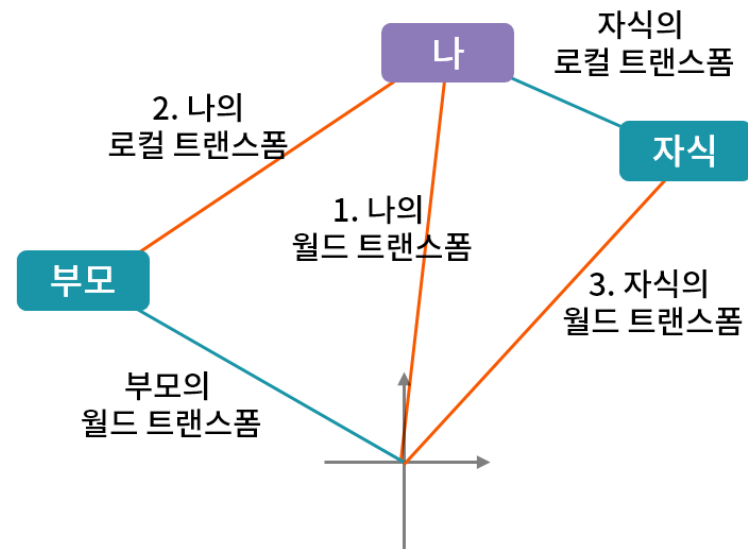
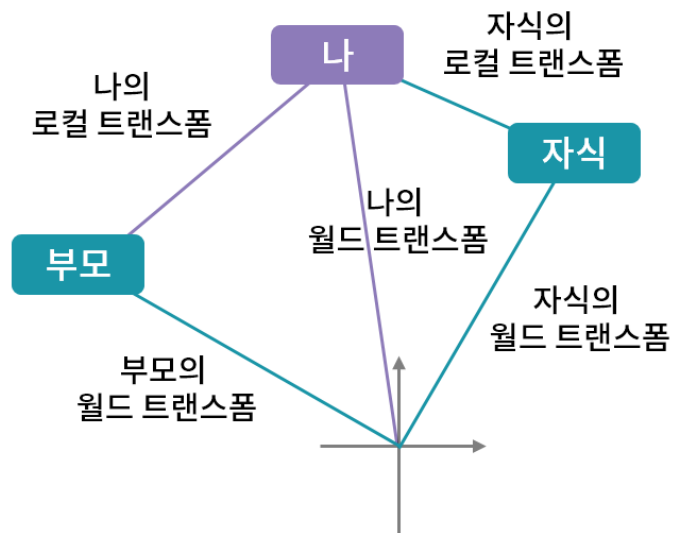


- 자신의 로컬 트랜스폼 변경



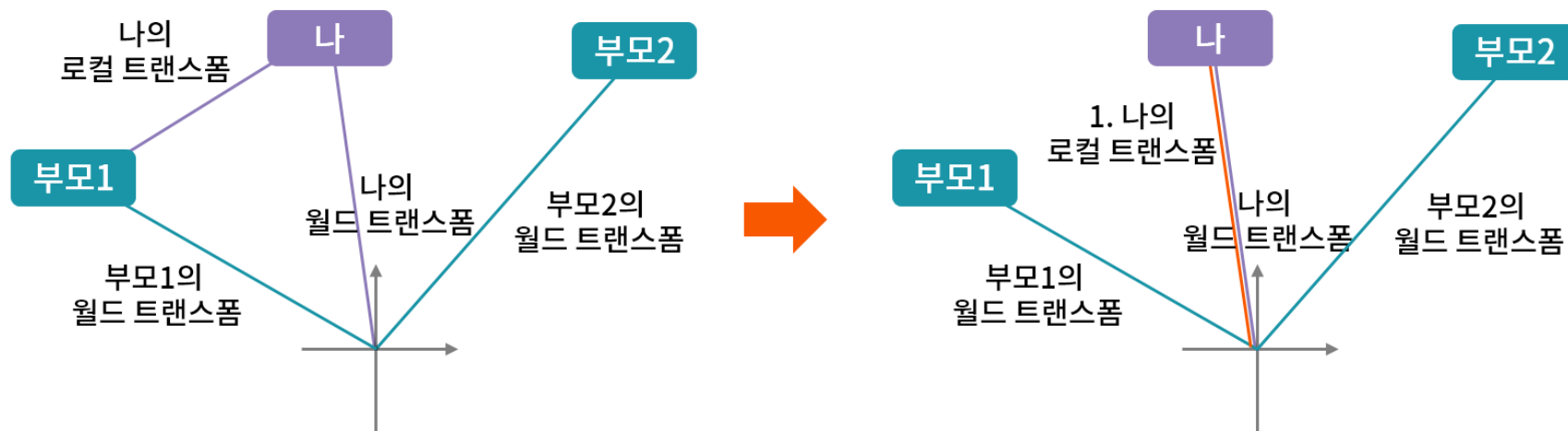


- 자신의 월드 트랜스폼 변경



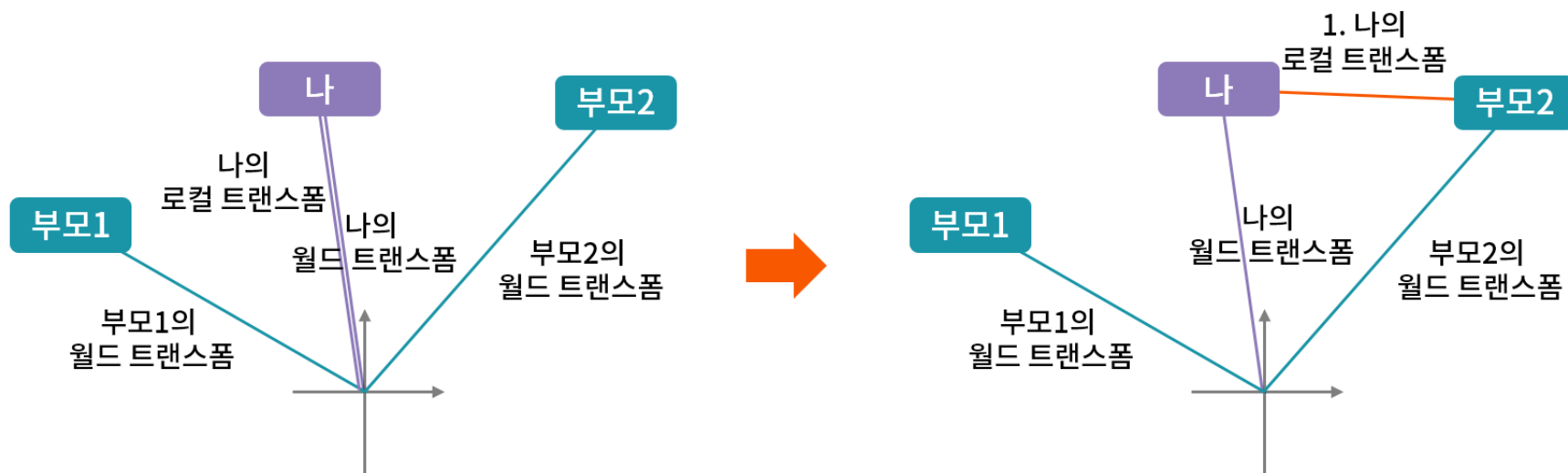


- 부모로부터 독립, 루트 트랜스폼으로 설정





- 새로운 부모에 편입





- 필수 기능

- ◆ 부모 월드 트랜스폼 정보 + 자신의 로컬 트랜스폼 정보 → 자신의 월드 트랜스폼 계산
- ◆ 부모 월드 트랜스폼 정보 + 자신의 월드 트랜스폼 정보 → 부모로부터 상대적인 자신의 로컬 트랜스폼 계산





- 부모 월드 크기 $\vec{s} = (s_x, s_y, s_z)$
- 자신 로컬 크기 $\vec{s}' = (s'_x, s'_y, s'_z)$
- 자신 월드 크기 $\vec{s}'_{world} = (s_x s'_x, s_y s'_y, s_z s'_z)$
- 두 벡터 곱 $\vec{v}_1 * \vec{v}_2 = (x_1, y_1, z_1) * (x_2, y_2, z_2) = (x_1 \cdot x_2, y_1 \cdot y_2, z_1 \cdot z_2)$
- 자신 월드 크기 $\vec{s}'_{world} = \vec{s} * \vec{s}'$
- 자신의 로컬 회전 q'
- 부모의 월드 회전 q
- 자신의 월드 회전 $q'_{world} = q' \cdot q$
- 부모 월드 트랜스폼 모델링 행렬

$$M_{parent} = \begin{bmatrix} x_x s_x & y_x s_y & z_x s_z & t_x \\ x_y s_x & y_y s_y & z_y s_z & t_y \\ x_z s_x & y_z s_y & z_z s_z & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$





- 자신의 로컬 트랜스폼 모델링 행렬

$$M_{local} = \begin{bmatrix} x'_x s'_x & y'_x s'_y & z'_x s'_z & t'_x \\ x'_y s'_x & y'_y s'_y & z'_y s'_z & t'_y \\ x'_z s'_x & y'_z s'_y & z'_z s'_z & t'_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$M_{world} = M_{parent} \cdot M_{local}$$

$$M_{world} = \begin{bmatrix} x_x s_x & y_x s_y & z_x s_z & t_x \\ x_y s_x & y_y s_y & z_y s_z & t_y \\ x_z s_x & y_z s_y & z_z s_z & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x'_x s'_x & y'_x s'_y & z'_x s'_z & t'_x \\ x'_y s'_x & y'_y s'_y & z'_y s'_z & t'_y \\ x'_z s'_x & y'_z s'_y & z'_z s'_z & t'_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$





$$\begin{aligned}\vec{t}'_{world} &= \left((x_x, y_x, z_x) \cdot (\vec{t}' * \vec{s}), (x_y, y_y, z_y) \cdot (\vec{t}' * \vec{s}), (x_z, y_z, z_z) \cdot (\vec{t}' * \vec{s}) \right) + \vec{t} \\ &= \left(\begin{bmatrix} x_x & y_x & z_x \\ x_y & y_y & z_y \\ x_z & y_z & z_z \end{bmatrix} \begin{bmatrix} t'_x s_x \\ t'_y s_y \\ t'_z s_z \end{bmatrix} \right) + \vec{t} \\ &= \left(q \cdot (\vec{t}' * \vec{s}) \right) + \vec{t}\end{aligned}$$





- $\vec{s} = (s_x, s_y, s_z)$ 역변환

$$\vec{s}^{-1} = \left(\frac{1}{s_x}, \frac{1}{s_y}, \frac{1}{s_z} \right) = \left(\begin{bmatrix} x_x & y_x & z_x \\ x_y & y_y & z_y \\ x_z & y_z & z_z \end{bmatrix} \begin{bmatrix} t'_x s_x \\ t'_y s_y \\ t'_z s_z \end{bmatrix} \right) + \vec{t}$$

- 크기 변환 없는 회전 q 역 변환

$$q^*$$

- 이동 트랜스 폼 역 변환 $\vec{t}^{-1}$

$$M^{-1} = (TRS)^{-1} = S^{-1}R^{-1}T^{-1}$$

$$S^{-1} = \begin{bmatrix} \frac{1}{s_x} & 0 & 0 & 0 \\ 0 & \frac{1}{s_y} & 0 & 0 \\ 0 & 0 & \frac{1}{s_z} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad R^{-1} = \begin{bmatrix} x_x & x_y & x_z & 0 \\ y_x & y_y & y_z & 0 \\ z_x & z_y & z_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad T^{-1} = \begin{bmatrix} 1 & 0 & 0 & -t_x \\ 0 & 1 & 0 & -t_y \\ 0 & 0 & 1 & -t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$





이동 트랜스폼 역 변환 $\vec{t}^{-1}$

$$M^{-1} = S^{-1}R^{-1}T^{-1} = \begin{bmatrix} \frac{x_x}{s_x} & \frac{x_y}{s_x} & \frac{x_z}{s_x} & \frac{-\hat{x} \cdot \vec{t}}{s_x} \\ \frac{y_x}{s_y} & \frac{y_y}{s_y} & \frac{y_z}{s_y} & \frac{-\hat{y} \cdot \vec{t}}{s_y} \\ \frac{z_x}{s_z} & \frac{z_y}{s_z} & \frac{z_z}{s_z} & \frac{-\hat{z} \cdot \vec{t}}{s_z} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\begin{aligned} \vec{t}^{-1} &= \left(\hat{x} \cdot \frac{-\vec{t}}{s_x}, \hat{y} \cdot \frac{-\vec{t}}{s_y}, \hat{z} \cdot \frac{-\vec{t}}{s_z} \right) \\ &= (\hat{x} \cdot -\vec{t}, \hat{y} \cdot -\vec{t}, \hat{z} \cdot -\vec{t}) * \vec{s}^{-1} \\ &= \left(\begin{bmatrix} x_x & x_y & x_z \\ y_x & y_y & y_z \\ z_x & z_y & z_z \end{bmatrix} \begin{bmatrix} -t_x \\ -t_y \\ -t_z \end{bmatrix} \right) * \vec{s}^{-1} \\ &= (q^* \cdot -\vec{t}) * \vec{s}^{-1} \end{aligned}$$





- 로컬 트랜스폼 = 부모 월드 트랜스폼 · 자신 월드 트랜스폼

$$M_{world} = M_{parent} \cdot M_{local}$$

$$M_{parent}^{-1} \cdot M_{world} = M_{parent}^{-1} \cdot M_{parent} \cdot M_{local}$$

$$M_{parent}^{-1} \cdot M_{world} = M_{local}$$

$$M_{local} = M_{parent}^{-1} \cdot M_{world}$$

$$\vec{s}^{-1} = \left(\frac{1}{s_x}, \frac{1}{s_y}, \frac{1}{s_z} \right) \quad q^* \quad \vec{t}^{-1}$$

$$\vec{s}'_{local} = \vec{s}^{-1} * \vec{s}' = \left(\frac{s'_x}{s_x}, \frac{s'_y}{s_y}, \frac{s'_z}{s_z} \right)$$

$$q'_{local} = q^* \cdot q'$$

$$\vec{t}'_{local} = q^* \cdot (\vec{t}' * \vec{s}^{-1}) + \vec{t}^{-1}$$

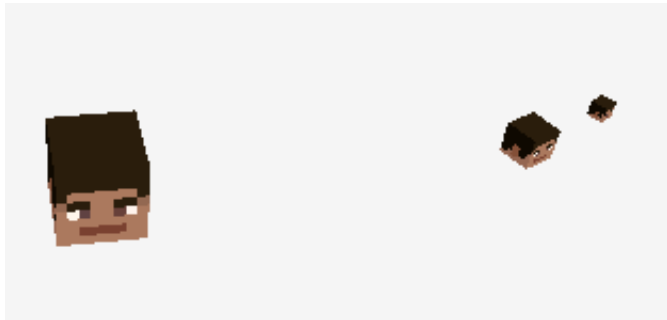




- 로컬 트랜스폼 = 부모 월드 트랜스폼 · 자신 월드 트랜스폼

```
Transform WorldToLocal(const Transform& ParentWorldTm) {  
    // 현재 트랜스폼 정보가 월드인 경우  
    Transform invPrn = ParentWorldTm.Inverse();  
  
    Transform ret;  
    ret.SetScale(invPrn.GetScale() * GetScale());  
    ret.SetRotation(invPrn.GetRotation() * GetRotation());  
  
    ret.SetPosition(  
        invPrn.GetPosition()  
        + (invPrn.GetScale()  
            * (invPrn.GetRotation()  
                * GetPosition()  
            )  
        )  
    );  
    return result;  
}
```





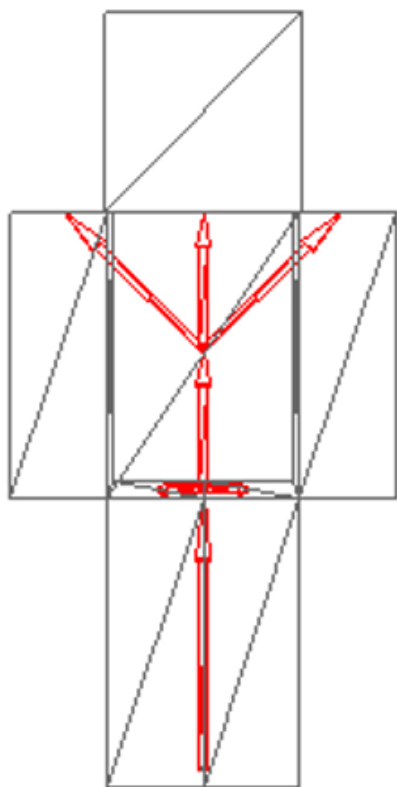
```
// 각 행성에 회전 부여
goSun.GetTransform().AddLocalYawRotation(rotateSpeedSun * InDeltaSeconds);
goEarth.GetTransform().AddLocalYawRotation(rotateSpeedEarth * InDeltaSeconds);
goMoon.GetTransform().AddLocalYawRotation(rotateSpeedMoon * InDeltaSeconds);

void TransformComponent::UpdateWorld() {
    // 자신의 월드 정보 업데이트
    if (HasParent()) {
        const TransformComponent& parent = *GetParentPtr();
        _WorldTransform = _LocalTransform.LocalToWorld(parent.GetWorldTransform());
    }
    else {
        _WorldTransform = _LocalTransform;
    }
    // 월드 정보 변경 시 자식의 월드 정보도 업데이트 ( 로컬 정보는 변함 없음. )
    UpdateChildrenWorld();
}
```





- 캐릭터 본의 계층 구조



Root

└ Pelvis

└ Spine

└ LeftArm

└ RightArm

└ Neck

└ LeftLeg

└ RightLeg





● 캐릭터 본 클래스

```
class Bone {  
public:  
    // 트랜스폼  
    TransformComponent& GetTransform() { return _Transform; }  
    const Transform& GetBindPose() const { return _BindPose; }  
    void SetParent(Bone& InBone) {  
        _ParentName = InBone.GetName();  
        _Transform.SetParent(InBone.GetTransform());  
    }  
  
    // 키 관련  
    bool HasParent() const { return _Name.compare(_ParentName) != 0; }  
    const std::string& GetParentName() const { return _ParentName; }  
  
private:  
    // 현재 트랜스폼 정보  
    TransformComponent _Transform;  
    // 최초 트랜스폼 정보  
    Transform _BindPose;  
    // 부모 정보  
    std::string _ParentName;
```





● 본 계층 구조 생성

```
// 본의 계층 구조 생성
Bone& root = characterMesh.GetBone(GameEngine::RootBone);
Bone& pelvis = characterMesh.GetBone(GameEngine::PelvisBone);
pelvis.SetParent(root);
Bone& spine = characterMesh.GetBone(GameEngine::SpineBone);
spine.SetParent(pelvis);
Bone& leftArm = characterMesh.GetBone(GameEngine::LeftArmBone);
leftArm.SetParent(spine);
Bone& rightArm = characterMesh.GetBone(GameEngine::RightArmBone);
rightArm.SetParent(spine);
Bone& leftLeg = characterMesh.GetBone(GameEngine::LeftLegBone);
leftLeg.SetParent(pelvis);
Bone& rightLeg = characterMesh.GetBone(GameEngine::RightLegBone);
rightLeg.SetParent(pelvis);
Bone& neck = characterMesh.GetBone(GameEngine::NeckBone);
neck.SetParent(spine);
```





● 리깅

```
// 메시에 리깅
static std::array<std::string, 6> boneOrder = {
    GameEngine::NeckBone, GameEngine::SpineBone, GameEngine::LeftArmBone, GameEn
    gine::RightArmBone, GameEngine::LeftLegBone, GameEngine::RightLegBone
};

cb.resize(v.size());
w.resize(v.size());
std::fill(cb.begin(), cb.end(), 1);

for (size_t part = 0; part < 6; part++) {
    Weight weight;
    weight.Bones = { boneOrder[part] };
    weight.Values = { 1.f };
    auto startIt = w.begin() + part * 24;
    std::fill(startIt, startIt + 24, weight);
}
```





```
// 애니메이션 로직을 담당하는 함수
void SoftRenderer::LateUpdate3D(float InDeltaSeconds) {
    GameObject& goPlayer = g.GetGameObject(PlayerGo);

    // 캐릭터 메시
    Mesh& m = g.GetMesh(goPlayer.GetMeshKey());

    // 목의 회전
    Bone& neckBone = m.GetBone(GameEngine::NeckBone);
    neckBone.GetTransform().SetLocalRotation(Rotator(neckCurve, 0.f, 0.f));

    // 팔의 회전
    Bone& leftArmBone = m.GetBone(GameEngine::LeftArmBone);
    leftArmBone.GetTransform().SetLocalRotation(Rotator(0.f, 0.f, armLegCurve));

    Bone& rightArmBone = m.GetBone(GameEngine::RightArmBone);
    rightArmBone.GetTransform().SetLocalRotation(Rotator(0.f, 0.f, -armLegCurve));

    // 다리의 회전
    Bone& leftLegBone = m.GetBone(GameEngine::LeftLegBone);
    leftLegBone.GetTransform().SetLocalRotation(Rotator(0.f, 0.f, -armLegCurve));

    Bone& rightLegBone = m.GetBone(GameEngine::RightLegBone);
    rightLegBone.GetTransform().SetLocalRotation(Rotator(0.f, 0.f, armLegCurve));
}
```





```
void SoftRenderer::DrawMesh3D() {
    std::vector<Vertex3D> vertices(vertexCount);
    std::vector<size_t> indice(InMesh.GetIndices());
    for (size_t vi = 0; vi < vertexCount; ++vi) {
        vertices[vi].Position = Vector4(InMesh.GetVertices()[vi]);

        // 위치에 대해 스킨링 연산 수행
        if (InMesh.IsSkinnedMesh()) {
            Vector4 totalPosition = Vector4::Zero;
            Weight w = InMesh.GetWeights()[vi];

            for (size_t wi = 0; wi < InMesh.GetConnectedBones()[vi]; ++wi) {
                std::string boneName = w.Bones[wi];

                if (InMesh.HasBone(boneName)) {
                    const Bone& b = InMesh.GetBone(boneName);
                    const Transform& t = b.GetTransform().GetWorldTransform(); // 모델링 공간
                    const Transform& bindPose = b.GetBindPose(); // 모델링 공간

                    // BindPose 공간을 중심으로 Bone의 로컬 공간을 계산
                    Transform boneLocal = t.WorldToLocal(bindPose);
                    // BindPose 공간으로 점을 변화
                    Vector3 localPosition = bindPose.WorldToLocalVector(vertices[vi].Position.ToVector3());
                    // BindPose 공간에서의 점의 최종 위치
                    Vector3 skinnedLocalPosition = boneLocal.GetMatrix() * localPosition;
                    // 모델링 공간으로 다시 변경
                    Vector3 skinnedWorldPosition = bindPose.GetMatrix() * skinnedLocalPosition;
                    // 가중치를 반영한 후 최종 위치에 누적
                    totalPosition += Vector4(skinnedWorldPosition, true) * w.Values[wi];
                }
            }
            vertices[vi].Position = totalPosition;
        }
    }
}
```





- 캐릭터 애니메이션
 - ◆ 캐릭터 애니메이션은 Bone Transform을 이용하여 Mesh 정점을 변형하는 기술
- Bone
 - ◆ 캐릭터 움직임의 기준 트랜스폼
- Bind Pose
 - ◆ 캐릭터의 초기 자세
 - ◆ 스키닝 계산의 기준 상태
- Weight
 - ◆ 정점이 각 Bone의 영향을 받는 비율
- Keyframe
 - ◆ 특정 시간의 Bone Transform 상태
- 트랜스폼 계층 구조
 - ◆ Bone은 부모-자식 관계의 트리 구조로 구성
 - ◆ 부모 Bone의 Transform은 자식 Bone의 World Transform에 반영
- 스키닝
 - ◆ 각 정점은 연결된 Bone의 Transform에 의해 변형
 - ◆ 여러 Bone이 연결된 경우 Weight를 이용해 변형 결과를 합산

